INFS2200/INFS7903 Assignment 2
Semester 2, 2026

Total Marks	30 marks (15% of the course)
Due Date	**4:00 pm, Friday 30th October 2026**
What to submit	Report file, and one SQL script file per question part
Where to submit	Report via Blackboard Ultra Course Website; SQL scripts via Gradescope
Assignment Version	1.0

The goal of the project assignments is to gain practical experience in applying several database management concepts and techniques using the PostgreSQL DBMS. In particular, this assignment mainly focuses on improving database efficiency with views, indexing and query planning, and on understanding what the DBMS does when transactions run concurrently.

Your main task is to first populate your database with the supplied data, then design, implement, and test the appropriate queries to perform the tasks explained in the next sections.

You must work on this project **individually**. Academic integrity policies apply. Please refer to *3.60.04 Student Integrity and Misconduct* of the University Policy for more information.

This document is in three sections: This section describes the database schema for the assignment and provides instructions on downloading the script file needed to create and populate the database. The middle section describes the tasks to be completed for this assignment. The final section describes the submission guidelines and marking scheme.

Enjoy the project!

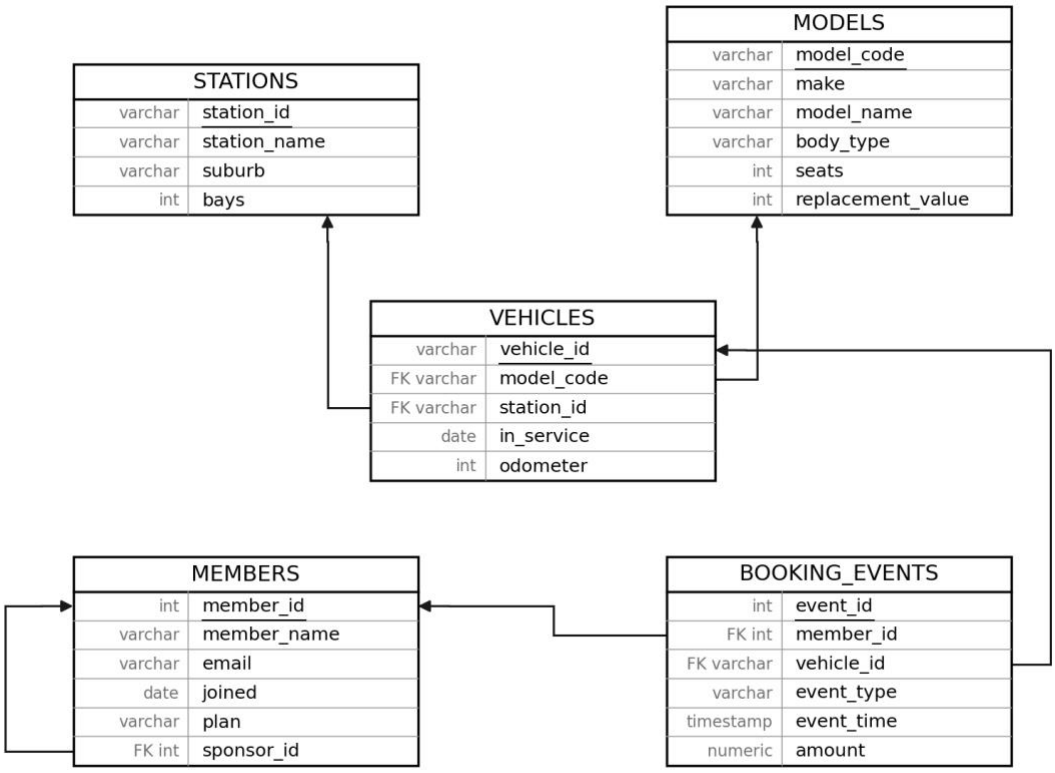
Section 1 - The FleetDB Database

You are a developer working on FleetDB.

Cooee Car Share is a member-owned car-sharing co-operative. Members reserve a vehicle online, collect it from a parking station, drive it, and return it to a station. FleetDB records the fleet, the membership, and every booking event. It has five tables:

- `STATIONS` records each parking station where vehicles are picked up and dropped off;
- `MODELS` records the specification shared by every vehicle of a given model, including its body type and replacement value;
- `VEHICLES` records each individual vehicle in the fleet, its model, and its home station;
- `MEMBERS` records each member of the co-operative. Members may be **sponsored** by another member — a business account sponsors a team account, which in turn sponsors the staff who drive. `sponsor_id` points to the sponsoring member, and is `NULL` for an account that pays its own way;
- `BOOKING_EVENTS` is an append-only log of everything that happens to a vehicle: `Reserve`, `Collect`, `Return`, `Cancel` and `Incident` events, each stamped with the member responsible.

`amount` holds the fare charged on a `Return` and the damage charge raised on an `Incident`; it is `NULL` for every other event type.



Entity-Relationship diagram

Entity-Relationship diagram

The tables are created as follows.

```
CREATE TABLE STATIONS (
    station_id    VARCHAR PRIMARY KEY,
    station_name  VARCHAR NOT NULL,
    suburb        VARCHAR NOT NULL,
    bays          INTEGER NOT NULL);

CREATE TABLE MODELS (
    model_code        VARCHAR PRIMARY KEY,
    make              VARCHAR NOT NULL,
    model_name        VARCHAR NOT NULL,
    body_type         VARCHAR NOT NULL,
    seats             INTEGER NOT NULL,
    replacement_value INTEGER NOT NULL);

CREATE TABLE VEHICLES (
    vehicle_id    VARCHAR PRIMARY KEY,
    model_code    VARCHAR NOT NULL REFERENCES MODELS,
    station_id    VARCHAR NOT NULL REFERENCES STATIONS,
    in_service    DATE NOT NULL,
    odometer      INTEGER NOT NULL);

CREATE TABLE MEMBERS (
    member_id     SERIAL PRIMARY KEY,
    member_name   VARCHAR NOT NULL,
    email         VARCHAR NOT NULL UNIQUE,
    joined        DATE NOT NULL,
    plan          VARCHAR NOT NULL,
    sponsor_id    INTEGER REFERENCES MEMBERS);

CREATE TABLE BOOKING_EVENTS (
    event_id      SERIAL PRIMARY KEY,
```

```
    member_id     INTEGER NOT NULL REFERENCES MEMBERS,
    vehicle_id    VARCHAR NOT NULL REFERENCES VEHICLES,
    event_type    VARCHAR NOT NULL,
    event_time    TIMESTAMP NOT NULL,
    amount        NUMERIC(8,2));
```

Notes on the schema

Two details of the schema will matter later:

1. `MEMBERS.sponsor_id` is a **self-referencing** foreign key, and sponsorship chains can be **more than one level deep**.
2. `MEMBERS.email` carries a `UNIQUE` constraint, so PostgreSQL has already built an index over it.

Further, the trigger `BI_COLLECT` and its associated user-defined function `UDF_BI_COLLECT` have been added to the `BOOKING_EVENTS` table. This trigger treats a vehicle's **most recent event** as its current state, and refuses to record a `Collect` that would break the co-operative's rules:

Most recent event for the vehicle	Meaning	A new `Collect` is …
`Return` or `Cancel`	available	allowed
`Reserve` by the same member	held for you	allowed
`Reserve` by a different member	held for someone else	**rejected**
`Collect`	already out on hire	**rejected**
`Incident`	off the road pending repair	**rejected**

No triggers implemented in Assignment 1 are present.

Setup

Before starting work on this assignment, run the setup file `a2-2026-fleetdb.sql` from `psql` with the `\i` command (it may take a minute):

```
\i a2-2026-fleetdb.sql
```

The script **drops and recreates** a database called `assignment2` and leaves you connected to it, so your work on this assignment is kept separate from your other work. Wait until you see the message `DONE! All data has been inserted.`

The setup file is available on Blackboard, and with `mget` as follows:

```
mget -O /infs2200/stor/data/assignments/2/a2-2026-fleetdb.sql
```

Note: as of this version of the assignment, the setup script version is 1.0

Section 2 - Assignment 2 Tasks

Q1 Views (4 marks total)

Q1.1 View A (1.5 marks)

The co-operative's planners want to know where each kind of vehicle is actually being picked up, so they can decide where to add parking bays.

(1) Create a view `V_BUSIEST_BAYS` that reports, **for each body type**, the single station at which vehicles of that body type have been collected the most times. The view should show the body type, the station name, and the number of collections, ordered from most collections to fewest. *(1 mark)*

> *Hint: this is a "top item within each group" problem. Consider a window function such as `RANK()` or `ROW_NUMBER()` with a `PARTITION BY` clause, and use Common Table Expressions to build the query up in a semantically meaningful way.*

(2) Query the view (`SELECT * FROM V_BUSIEST_BAYS;`) and report the results. Report and explain the query plan. You should refer to the reported times, costs, the number of rows fetched, your knowledge of query planning (and anything else you feel is relevant). *(0.5 marks)*

Q1.2 View B (1.5 marks)

Every charge a member incurs is ultimately billed to the account at the **top** of their sponsorship chain. A member with no sponsor pays their own charges. A sponsored member's charges pass to their sponsor — and if that sponsor is itself sponsored, the charges keep passing up the chain until they reach an account with no sponsor.

Create a view `V_ACCOUNT_LIABILITY` that identifies the **ten accounts liable for the largest total amount** charged between 1 January 2026 and 30 June 2026 inclusive. The view should show the account's member id, member name, and the total amount billed to it.

> *Hint: the chain can be more than one level deep, so an expression such as `COALESCE(sponsor_id, member_id)` will give the wrong answer. Consider a `WITH RECURSIVE` Common Table Expression.*

Q1.3 Materialised view (1 mark)

(1) Create a materialised view using the same query as `V_ACCOUNT_LIABILITY`, called `MV_ACCOUNT_LIABILITY`. *(0 marks)*

(2) Report, compare and explain the following two queries, which select the contents of the view and the materialised view respectively. *(0.5 marks)*

Query A:

```
SELECT * FROM V_ACCOUNT_LIABILITY;
```

Query B:

```
SELECT * FROM MV_ACCOUNT_LIABILITY;
```

You should refer to the reported times, costs, the number of rows fetched, your knowledge of query planning (and anything else you feel is relevant).

(3) A member logs a large damage charge. Run the following statement, then re-run Query A and Query B and report what each returns. Explain the difference, then issue `REFRESH MATERIALIZED VIEW MV_ACCOUNT_LIABILITY;` and re-run Query B. In one or two sentences, state the trade-off a designer accepts when choosing a materialised view over an ordinary view. *(0.5 marks)*

```
INSERT INTO BOOKING_EVENTS (member_id, vehicle_id, event_type, event_time, amount)
SELECT member_id, 'VEH-0001', 'Incident', TIMESTAMP '2026-05-04 09:15:00', 90000.00
FROM MEMBERS WHERE sponsor_id IS NOT NULL ORDER BY member_id DESC LIMIT 1;
```

Q2 Indexes and performance (8 marks total)

Q2.1 Composite index (3 marks)

(1) Create an index `IDX_EVENT_TYPE_TIME` on the `event_type` and `event_time` fields (in that order) of the `BOOKING_EVENTS` table. *(0.5 marks)*

(2) Re-run your query from Q1.1 (`SELECT * FROM V_BUSIEST_BAYS;`). Report the query plan. Compare this query plan to the one from Q1.1 and explain the differences, if any. *(1.5 marks)*

(3) Now report and explain the query plan for the query below, which counts the incidents logged in March 2026. Explain why the planner makes different use of `IDX_EVENT_TYPE_TIME` here than it did in part (2). *(1 mark)*

```
SELECT count(*), sum(amount) FROM BOOKING_EVENTS
WHERE event_type = 'Incident'
  AND event_time >= TIMESTAMP '2026-03-01'
  AND event_time <  TIMESTAMP '2026-04-01';
```

Q2.2 Function-based index (5 marks)

The co-operative wants to know which e-mail providers its members use, so that it can investigate why its newsletter is being rejected by some of them. The **provider domain** is the part of the e-mail address after the `@` character. Members have entered their addresses inconsistently, so domains must be compared **case-insensitively** — `Bigpond.com` and `bigpond.com` are the same provider.

(1) Write an expression that identifies each member's e-mail provider domain, and use it to report each distinct domain together with the number of members who use it, most popular first. You may use string processing functions, including the regex functions. *(1 mark)*

(2) Report and explain the query plan for the query in part (1). You should refer to the reported times, costs, the number of rows fetched, your knowledge of query planning (and anything else you feel is relevant). *(1 mark)*

(3) `MEMBERS.email` already carries a `UNIQUE` constraint, and therefore already has an index on it. In one or two sentences, explain why that existing index cannot answer a query about provider domains. *(1 mark)*

(4) Create a function-based index called `IDX_MEMBER_DOMAIN` to potentially speed up queries on the expression from part (1). *(0.5 marks)*

(5) Report the query plan, with the function-based index in place, for **both** of the following:

- the query you wrote in part (1); and
- a lookup of the members at one specific, rare provider:

```
SELECT member_id, member_name, email FROM MEMBERS
WHERE <your expression> = 'exemail.com.au';
```

Explain the differences to the previous plans, if any, and why the index helps one of these queries much more than the other. You should refer to the reported times, costs, the number of rows fetched, your knowledge of query planning (and anything else you feel is relevant). *(1.5 marks)*

> **Bonus content:** an index built over an expression only helps when a query contains *that same expression*. This is why production systems often store a derived value (such as a normalised domain) in its own column — it can then be indexed, constrained and queried directly.

Q3 Query formulation and query planning (7 marks total)

Q3.1 Two ways to ask the same question (3 marks)

Some vehicles in the fleet have never actually been driven by a member — they may be new stock, or only ever reserved and then cancelled.

(1) Write **two different** SQL statements that both report the `vehicle_id` of every vehicle that has **never** been the subject of a `Collect` event: (A) using `NOT EXISTS`; (B) without using `NOT EXISTS` or `NOT IN` — for example with an outer join. Both statements must return the same rows. *(1.5 marks)*

(2) Report and compare the execution plans of statements (A) and (B). Are they the same plan? Explain what the planner has done and why. *(1.5 marks)*

Q3.2 Suppressing scan types (4 marks)

PostgreSQL allows you to influence the query planner with runtime configuration parameters in `psql`.

https://www.postgresql.org/docs/17/runtime-config-query.html

In this section we will suppress the use of several different scan types in order to compare queries. Suppressing plan types is not recommended for production, but is useful for understanding why one plan would be chosen over another.

Report and explain the execution plan for the following two queries (C & D):

3. with both index and sequential scans enabled;
4. with index scans enabled and sequential scans suppressed;
5. with index scans suppressed and sequential scans enabled.

Explain why the query planner might choose one method over another. You should refer to the reported times and costs from the execution plan, the number of rows fetched, your knowledge of query planning (and anything else you feel is relevant).

Note: For this question, bitmap and index-only scans count as index scans too. The index `IDX_EVENT_TYPE_TIME` from Q2.1 should still be in place.

Query C:

```
SELECT * FROM BOOKING_EVENTS WHERE event_type < 'Collect';
```

Query D:

```
SELECT * FROM BOOKING_EVENTS WHERE event_type >= 'Collect';
```

(2 marks for explaining up to 6 possible query plans; 2 marks for explaining the query planner's choices.)

Between them, Queries C and D return every row of `BOOKING_EVENTS` exactly once — they differ only in which side of `'Collect'` they take. Your explanation should account for why that is enough to change the plan.

Be sure to reset index, bitmap, index-only and sequential scans back to `on` before proceeding to another question.

Q4 Transactions (9 marks total)

In PostgreSQL, a transaction is set up by surrounding the SQL commands of the transaction with `BEGIN` and `COMMIT` commands. For example, a banking transaction would look like:

```
BEGIN;
UPDATE accounts SET balance = balance - 100.00
  WHERE name = 'Alice';
-- some more statements etc etc
COMMIT;
```

https://www.postgresql.org/docs/current/tutorial-transactions.html

In this question we will examine what happens when two transactions occur concurrently.

Background: how PostgreSQL reads

You have seen two ways of keeping a concurrent schedule serialisable: locking protocols such as two-phase locking, and timestamp ordering. **PostgreSQL uses neither of these for ordinary reads**, so what you are about to observe will not follow from either one directly. This section gives you what you need; you are not expected to have met it before.

Instead of taking read locks, a PostgreSQL query reads from a **snapshot** — a consistent picture of the database containing exactly those changes that were committed before the snapshot was taken. Because reading acquires no locks, readers never block writers and writers never block readers.

At PostgreSQL's default isolation level, `READ COMMITTED`, **a new snapshot is taken at the start of every statement**. Two statements in the same transaction may therefore see different data, if another transaction committed in between them.

Two consequences are worth holding on to as you work through this question:

- **A trigger runs inside the transaction that fired it.** Any query the trigger makes is answered from that transaction's current snapshot. It gets no special or privileged view of the database.
- **A snapshot is not a lock.** Reading a row records nothing and reserves nothing; it does not prevent another transaction from changing what you just read.

Q4.1 A race for the same car (6 marks)

Two members go after the same car at the same moment:

- **Session 1** is a member at the parking bay about to drive away. It checks that the vehicle is available, and then records a `Collect`.
- **Session 2** is a member on the website. It checks that the same vehicle is available, and then records a `Reserve` for three days' time, and commits — all while session 1 is still open.

You will need two `psql` connections open side by side, both at the default isolation level. You can use the `now()` function to provide a timestamp. **Include screenshots of both connections.**

(1) In your first `psql` connection, begin a transaction and then perform a query to identify a vehicle which is **currently available** — that is, a vehicle whose most recent event is a `Return`. Do not commit yet. *(2 marks)*

(2) In your *second* connection, begin *another* transaction. Run the same availability check for the vehicle session 1 picked, then record a `Reserve` on it for some **other** member, 3 days in the future, and commit. *(0.5 marks)*

(3) Back in your first connection, attempt to record a `Collect` on that vehicle for a **different** member at the present time, then commit. Report what happened at each step, and which event or events ended up recorded against the vehicle. *(1 mark)*

(4) Explain the outcome, both in database terms and in car-share terms. You may include a schedule or precedence graph of the transaction events if it aids your explanation. *(2.5 marks)*

> *Hint: was the answer to the query in step (1) still true by the time step (3) ran? Which snapshot did the `BI_COLLECT` trigger use when it ran? And what, if anything, stopped session 2 from changing the answer underneath session 1?*

Q4.2 Deadlock (3 marks)

In Q4.1 neither transaction could make progress on the other's terms, but only one of them was ever blocked. A **deadlock** is the symmetric case: two transactions each hold something the other is waiting for, so neither can move. Neither has done anything wrong on its own — it is the interleaving that traps them.

Two staff are correcting odometer readings after a service run, and they work through the same two vehicles in opposite orders.

(1) In two `psql` connections, begin a transaction in each. In your first connection, update the odometer of `VEH-0401`, and then the odometer of `VEH-0501`. In your second connection, update those same two vehicles in the **opposite order**. Run the statements one at a time so you can see which one blocks. Report what happened at each step, quoting any error message in full. *(1.5 marks)*

(2) Draw the wait-for graph as it stood immediately before the DBMS intervened, and mark the cycle on it. State what the DBMS did about the deadlock, and what became of each of the two transactions. *(1 mark)*

(3) Name **one** strategy that would have *prevented* this deadlock rather than detecting it after the fact, and say what that strategy would cost the co-operative. *(0.5 marks)*

> *Hint: an `UPDATE` takes a lock on the row it changes and holds it until the transaction ends. Which statement in each session was still waiting when the error appeared?*

Section 3 - Deliverables and Marking

There are 28 marks for functionality, understanding and explanations, and 2 marks for presentation.

Submission

Submit the following by the due date (**4:00 pm, Friday 30th October 2026**).

Late submissions will be penalised unless you are approved for an extension (refer to Section 5.3 of the ECP).

6. **A report file** titled `4xxxxxxx_A2.pdf` in PDF format, containing relevant code, screenshots and answers to all questions. Submit via the Turnitin link "Report Submission" on the Blackboard Ultra Course Website.
7. **A set of SQL script files**, one per question part, submitted via the Gradescope link "SQL Script Submission". Each file must be plain text with a `.sql` extension, and must be named `4xxxxxxx_A2_<question>.sql`, where `<question>` is the number of the question part it answers. Your answer to Q2.1 (3), for instance, goes in `4xxxxxxx_A2_2.1.3.sql`.

Replace `4xxxxxxx` with your student number in every filename. Upload the script files individually, or as a single `.zip` archive containing them with no enclosing folder.

Which script files to submit

Submit the twenty files listed below. The remaining question parts — Q2.2 (3), Q4.1 (4), Q4.2 (2) and Q4.2 (3) — ask only for an explanation and have no script file; answer those in your report.

File	What it should contain
`4xxxxxxx_A2_1.1.1.sql`	The `CREATE VIEW V_BUSIEST_BAYS` statement
`4xxxxxxx_A2_1.1.2.sql`	`SELECT * FROM V_BUSIEST_BAYS;` and the `EXPLAIN` you ran on it
`4xxxxxxx_A2_1.2.sql`	The `CREATE VIEW V_ACCOUNT_LIABILITY` statement
`4xxxxxxx_A2_1.3.1.sql`	The `CREATE MATERIALIZED VIEW MV_ACCOUNT_LIABILITY` statement
`4xxxxxxx_A2_1.3.2.sql`	Queries A and B, with the `EXPLAIN` you ran on each
`4xxxxxxx_A2_1.3.3.sql`	The supplied `INSERT`, your re-runs of Queries A and B, and the `REFRESH`
`4xxxxxxx_A2_2.1.1.sql`	The `CREATE INDEX IDX_EVENT_TYPE_TIME` statement
`4xxxxxxx_A2_2.1.2.sql`	The re-run of your Q1.1 query, with its `EXPLAIN`
`4xxxxxxx_A2_2.1.3.sql`	The `EXPLAIN` of the March incidents query
`4xxxxxxx_A2_2.2.1.sql`	Your statement reporting the count of members per provider domain
`4xxxxxxx_A2_2.2.2.sql`	The `EXPLAIN` of your Q2.2 (1) statement
`4xxxxxxx_A2_2.2.4.sql`	The `CREATE INDEX IDX_MEMBER_DOMAIN` statement
`4xxxxxxx_A2_2.2.5.sql`	Both `EXPLAIN`s, run with the function-based index in place
`4xxxxxxx_A2_3.1.1.sql`	Statements (A) and (B)
`4xxxxxxx_A2_3.1.2.sql`	The `EXPLAIN` of statement (A) and of statement (B)
`4xxxxxxx_A2_3.2.sql`	Queries C and D under all three settings, including the `SET` statements
`4xxxxxxx_A2_4.1.1.sql`	Connection 1: `BEGIN` and your availability query
`4xxxxxxx_A2_4.1.2.sql`	Connection 2: `BEGIN`, the availability query, the `Reserve`, `COMMIT`
`4xxxxxxx_A2_4.1.3.sql`	Connection 1: the attempted `Collect`, and `COMMIT`
`4xxxxxxx_A2_4.2.1.sql`	Every statement you ran in both connections, in the order you ran them

How your script files will be run

You will be marked primarily on the contents of your report file. However, the teaching team must be able to reproduce the screenshots in your report by loading `a2-2026-fleetdb.sql` into a fresh database and then running your files **in question order**, each with the `\i` command in `psql`. Write your files on that assumption: a file may rely on objects created by an earlier file, but no file should repeat the dataset setup.

A single script cannot show two connections running at the same time, so in the Q4 files mark each statement with the connection you ran it in, and keep the statements in the order you actually ran them:

```
-- connection 1
BEGIN;
...
-- connection 2
BEGIN;
...
```

If code is not provided for any question such that your screenshots cannot be reproduced, your marks will be halved for that question. A file that is misnamed, empty, or does not run counts as not provided.

Report File & Presentation (2 marks)

Your report file should include the following content:

- Answers to all the questions in the Assignment Tasks section.
- If you are asked to write SQL statements, you need to include those statements in your report.
- After you execute a SQL statement, if PostgreSQL produces any output (e.g. query result, query execution time, query plan, etc), you should also include a screenshot of the output as well.
    - Where one sub-question asks you to write a query and a later sub-question asks you to report on its timing, please simply refer back to the initial screenshot.

The presentation mark is given as follows:

- Ordinary query results (not plans) do not exceed a page — any query returning more than 20 rows may show just the first 10 and the last 10 rows. You may find some helpful instructions for formatting query output in Practical 1. *(1 mark)*
- No result tuples are broken across lines, and all query plans are shown in full (if a plan takes more than one page, break it at an appropriate spot). *(1 mark)*

Marking Scheme

Question	Marks	Specific Criteria
1.1 (1)	1	`V_BUSIEST_BAYS` created with the correct name and semantics: one row per body type, naming the station with the most collections. Tested by `SELECT * FROM V_BUSIEST_BAYS;`
1.1 (2)	0.5	Result and query plan reported and explained with reference to costs, times and row counts.

1.2	1.5	`V_ACCOUNT_LIABILITY` created with the correct name and semantics. Full marks require the charges to be rolled up the **whole** sponsorship chain; a one-level rollup earns at most 0.5.
1.3 (2)	0.5	Both plans reported and compared; the cost of the view versus the stored result is explained.
1.3 (3)	0.5	Staleness of the materialised view demonstrated and the trade-off stated.
2.1 (1)	0.5	Index created with the correct name and semantics.
2.1 (2)	1.5	Plan reported and compared with Q1.1; correctly explains why the index is (or is not) used.
2.1 (3)	1	Plan reported; the role of selectivity and of the second index column is explained.
2.2 (1)	1	One SQL statement producing the correct per-domain counts, folding case.
2.2 (2)	1	Plan reported and explained.
2.2 (3)	1	Correctly explains that an index on `email` orders whole addresses, not domains.
2.2 (4)	0.5	Function-based index created with the correct name and semantics.
2.2 (5)	1.5	Both plans reported; the difference between the aggregate and the selective lookup is explained.
3.1 (1)	1.5	Two correct statements returning the same rows, using the required formulations.
3.1 (2)	1.5	Plans reported and compared; anti-join behaviour explained.
3.2 (1)	2	Each distinct query plan explained, for all combinations of query and settings.
3.2 (2)	2	The planner's decisions explained by comparing plans; selectivity identified as the cause.
4.1 (1)	2	Availability query correct — must find a vehicle whose *latest* event is a Return, not merely one that has a Return.
4.1 (2)	0.5	Second transaction carried out correctly and screenshotted.
4.1 (3)	1	Outcome of each step reported, and the resulting events

		identified.
4.1 (4)	2.5	Outcome explained in database terms (the step (1) answer went stale; the trigger re-read on a later snapshot) and in car-share terms. A schedule or precedence graph earns credit but is not required.
4.2	3	Deadlock produced and the error quoted (1.5); wait-for graph drawn with the cycle marked, and the DBMS's response described (1); a prevention strategy named with its cost (0.5).
P	2	Presentation marks as described above.
Total	**30**	

Enjoy the project! Good luck!

END OF ASSIGNMENT 2